



Hotel Room Booking System

1. Cover Page

B.Tech Computer Science Engineering

Subject: Backend Development – Node.js, Express.js & MongoDB

Project Title: Hotel Room Booking System

Case Study: 126

Name: Manan Jain

Roll No.: 150096725166

Cohort: Sam Altman

2. Abstract

The Hotel Room Booking System is a full-stack web application that lets guests search hotel rooms and book them online, while a hotel manager controls the room inventory and pricing. It is built as Case Study 126 for the Backend Development subject.

The backend is a REST API built with Node.js, Express.js and Mongoose, storing data in MongoDB Atlas. The frontend is a React application (built with Vite) that talks to the API using the Fetch API. Users register and log in; passwords are hashed with bcryptjs and sessions use JSON Web Tokens (JWT).

Guests can filter rooms by type and maximum price, sort by price, browse paginated results, book a room for a date range, and view or cancel their bookings. Managers can add rooms, update them through the API, deactivate them, and view all bookings.

Double booking is prevented in the backend. Before a booking is saved, the server checks whether any confirmed booking for the same room overlaps the requested dates. If one does, the request is rejected with a 409 Conflict response.

3. Introduction and Problem Statement

The Hotel Room Booking System lets guests search for rooms and book them online, while the hotel manager controls room inventory and pricing. It replaces manual booking, which is slow and can cause double booking: two guests promised the same room for the same night. Here the availability check runs on the server, so it cannot be bypassed from the browser. In the interface the hotel appears under the name “Manan Palace”.

The official Case Study 126 states:

A hotel wants guests to search room availability by date and book rooms online, while the hotel manager manages room inventory and pricing. The listing should support filtering by room type and price, with pagination, and a room cannot be double-booked for the same dates.

The expected outcomes and deliverables of the case study are mapped to the implementation in Section 16.

4. Objectives and Scope

Objectives (from the case study) and how each was met:

1. Design Room and Booking schemas using Mongoose: Room, Booking and Guest models with validation and references.
2. CRUD for room inventory: create, read, update and deactivate through the API; the dashboard screen offers add and deactivate.
3. Filtering, sorting and pagination: type, maxPrice, sort, page and limit on GET /api/rooms.
4. Date-based availability checking before booking: done inside POST /api/bookings. The room search itself does not take dates (see Section 16).
5. Role-based authorization for manager-only inventory control: protect and restrictTo('manager') middleware.

Scope: what the current system supports

- **Guests:** register and log in; browse active rooms with filters, sorting and pages; open a room's details; book for chosen dates; view and cancel their own bookings.
- **Managers:** log in to a protected dashboard; add and deactivate rooms; view all bookings. Room updates (including price) and cancelling any booking are available through the API.
- **Authentication:** email and password with JWT. Public registration always creates a guest; the manager account is created by a seed script.
- **Not included:** online payment, email notifications, image upload, and a date filter on room search.

5. Technologies Used

Only technologies present in the project's `package.json` files or source code are listed.

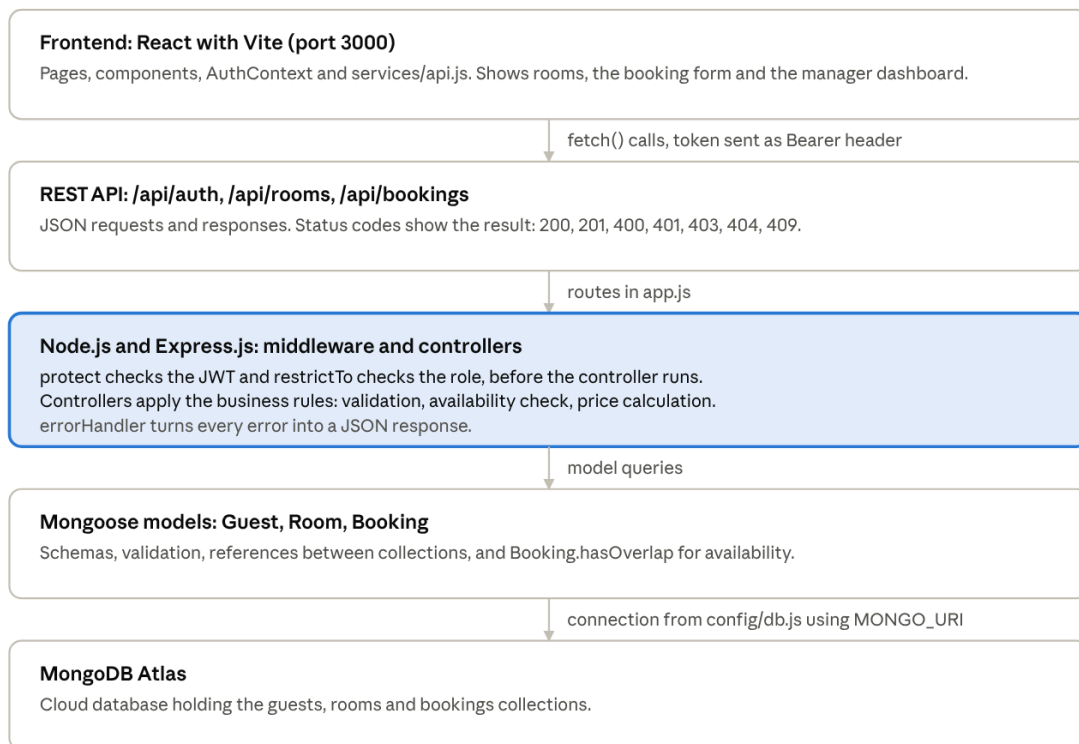
Technology	Purpose
Node.js, Express.js (v5)	Backend runtime and web framework (routing, middleware, REST API)
MongoDB Atlas	Cloud-hosted database
Mongoose (v9)	Schemas, validation, queries and references
jsonwebtoken	Issuing and verifying login tokens (JWT)
bcryptjs	Password hashing and comparison
cors, dotenv, nodemon	Cross-origin access, <code>.env</code> configuration, auto-restart in development
Jest, Supertest, mongodb-memory-server	Automated API tests with an in-memory database
React (v19), Vite	User interface and build / dev server (proxies <code>/api</code> to the backend)
React Router (v7)	Page navigation and protected routes

Technology	Purpose
Fetch API	Frontend calls to the REST API (Axios is not used)
Postman	Collections for testing the API

6. System Architecture

A request travels down the layers and the response returns up. The frontend holds no business rules: validation, authorization and the availability check are all in the Express middleware and controllers.

Every request passes through Express middleware and controllers before it reaches MongoDB



system architecture · 5 layers

7. Project Structure

The ZIP contains backend and frontend folders. Only files that exist are listed.

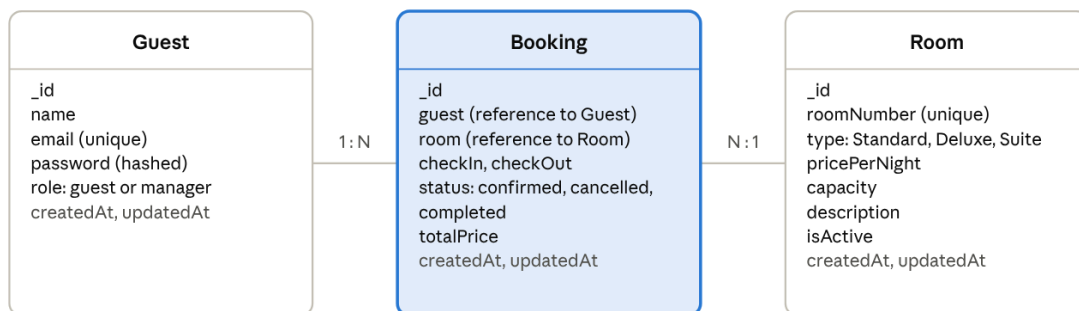
File / Folder	Purpose
backend/server.js	Entry point: loads .env, connects to MongoDB, starts the server
backend/app.js	Express app: CORS, JSON parsing (10 kb limit), /health, route mounting, 404 and error handler
backend/config/db.js	MongoDB Atlas connection using MONGO_URI
backend/models/	Guest.js, Room.js, Booking.js (includes the hasOverlap check)
backend/controllers/	authController.js, roomController.js, bookingController.js: business logic
backend/routes/	authRoutes.js, roomRoutes.js, bookingRoutes.js: URLs, middleware attached
backend/middleware/	auth.js (protect, restrictTo) and errorHandler.js
backend/utils/	AppError.js (error with status code), asyncHandler.js (forwards async errors)
backend/scripts/seedManager.js	One-time script that creates the manager account
backend/tests/, backend/postman/	Jest tests, integration script, three Postman collections
backend/.env.example, README.md	Configuration template (names only) and backend documentation
frontend/src/pages/	HomePage, RoomsPage, RoomDetailPage, LoginPage, RegisterPage, MyBookingsPage, ManagerDashboard

File / Folder	Purpose
frontend/src/components/	Navbar, Footer, RoomCard, SearchFilterBar, BookingModal, ProtectedRoute
frontend/src/context/AuthContext.jsx	Login state shared across the app
frontend/src/services/api.js	All backend calls (Fetch) and the currency formatter
frontend/src/App.jsx, main.jsx, vite.config.js	Routes, app start, dev server on port 3000 with /api proxy

8. Database Design

Three Mongoose models are stored as three MongoDB collections, all with timestamps (createdAt, updatedAt).

One guest and one room can each have many bookings



N means many. Each booking stores one guest id and one room id.

Guest, Booking and Room relationship · 3 collections

Model	Important fields and validation
Guest	name, email (valid format, unique, lowercase), password (minimum 8 characters, hashed, hidden from queries), role (guest by default, or manager)

Model	Important fields and validation
Room	roomNumber (unique), type (Standard, Deluxe, Suite), pricePerNight (at least 1), capacity (whole number, at least 1), description, isActive (default true)
Booking	room and guest (required ObjectId references), checkIn, checkOut (must be later than check-in), status (confirmed default, cancelled, completed), totalPrice (set by the server)

Relationships. One guest can have many bookings and one room can have many bookings; each booking belongs to one guest and one room. The API uses `populate` to return room and guest details with a booking. `isActive` is the room status: `false` hides the room from guests and blocks new bookings. Rooms are deactivated, not deleted, so bookings never point to a missing room. Booking has indexes on `room + checkIn + checkOut` (overlap queries) and `guest + status` (my bookings).

9. Authentication and Authorization

Authentication answers “Who is the user?”. **Authorization** answers “What is the user allowed to do?”.

- **Registration** (POST `/api/auth/register`): name, email and password are required; a duplicate email returns 409. The role is always saved as `guest`, whatever the client sends, so nobody can register as a manager.
- **Password hashing:** a pre-save hook in the `Guest` model hashes passwords with `bcryptjs` (12 salt rounds). The password is hidden from queries and removed from every JSON response.
- **Login** (POST `/api/auth/login`): the entered password is compared with the stored hash. A wrong email or password gives the same 401 message.
- **JWT:** on login or registration the server returns a token holding the user id and role, valid for the time in `JWT_EXPIRES_IN` (default 7 days). The frontend sends it as `Authorization: Bearer <token>`.
- **Authentication middleware protect:** verifies the token, loads the user from the database (so a deleted user's token stops working) and sets `req.user`.

- **Authorization middleware restrictTo:** compares `req.user.role` with the allowed roles and returns 403 otherwise, for example `router.post('/', protect, restrictTo('manager'), createRoom)`.
- **Manager account:** created by `npm run seed:manager` (`scripts/seedManager.js`), which reads its details from environment variables.

Role	Allowed actions
Guest	Search rooms; create bookings; view and cancel own bookings
Manager	All guest actions, plus create, update and deactivate rooms; view all bookings; view or cancel any booking

10. Room Management

Operation	Endpoint	Access
Search / list active rooms	GET <code>/api/rooms</code>	Public
View one room	GET <code>/api/rooms/:id</code>	Public
Create	POST <code>/api/rooms</code>	Manager
Update (price, type, etc.)	PUT <code>/api/rooms/:id</code>	Manager
Deactivate (soft delete)	DELETE <code>/api/rooms/:id</code>	Manager

- **Create** needs `roomNumber`, `type`, `pricePerNight` and `capacity` (description optional); a duplicate room number returns 409.
- **Update** accepts only `roomNumber`, `type`, `pricePerNight`, `capacity`, `description` and `isActive`, and schema validators still run.
- **Deactivate** sets `isActive: false`. A room with confirmed bookings cannot be deactivated (409); the manager must cancel those bookings first.

Search, filter, sort and pagination (GET `/api/rooms`, active rooms only):

Parameter	Behaviour
<code>type</code>	Standard, Deluxe or Suite; anything else returns 400

Parameter	Behaviour
maxPrice	Rooms with pricePerNight up to this value (must be 1 or more)
sort	pricePerNight (low to high) or -pricePerNight; default is createdAt
page, limit	Defaults 1 and 10; limit is capped at 50

Example: GET

/api/rooms?type=Deluxe&maxPrice=5000&sort=pricePerNight&page=1&limit=6.

The response has data, results and pagination (total, page, limit, totalPages).

11. Booking Management and Double-Booking Prevention

All booking routes require login. POST /api/bookings performs these steps:

1. The guest sends roomId, checkIn and checkOut; missing values or a malformed room id return 400.
2. Dates must be valid, check-in must not be in the past, and check-out must be after check-in (400 otherwise).
3. The room must exist and be active (404 otherwise).
4. A Mongoose transaction starts and Booking.hasOverlap checks for conflicting confirmed bookings.
5. If a conflict exists, the server returns 409 and saves nothing.
6. Otherwise it calculates totalPrice = nights × pricePerNight and saves the booking with the logged-in guest and status confirmed. The client cannot set price or status.
7. The booking is returned with room and guest details (201).

Other operations: GET /api/bookings/my (own bookings), GET /api/bookings (all for a manager, own for a guest), GET /api/bookings/:id (owner or manager, otherwise 403) and PATCH /api/bookings/:id/cancel (owner or manager; DELETE is an alias). The schema allows confirmed, cancelled and completed; the code sets only the first two.

The overlap rule. A room cannot be booked if a confirmed booking for the same room overlaps the requested dates. Booking.hasOverlap counts bookings matching:

```
{
  room: roomId,
```

```

    status: 'confirmed',
    checkIn: { $lt: requestedCheckOut },    // existing stay starts
before the new one ends
    checkOut: { $gt: requestedCheckIn }      // existing stay ends after
the new one starts
}

```

If the count is above zero, the dates overlap.

Example. Existing booking: 10 October to 15 October. Requested: 12 October to 14 October.

Condition	Check	Result
Existing check-in before requested check-out	10 Oct < 14 Oct	True
Existing check-out after requested check-in	15 Oct > 12 Oct	True

Both are true, so the server rejects the request with 409 Conflict. A request for 15 to 17 October is accepted, because $15 > 15$ is false: a new guest may check in on the day the previous guest checks out.

Notes. Cancelled bookings do not block dates. If the database does not support transactions (for example a standalone local MongoDB), the code runs the same check without one. The overlap rule lives in application code, not as a database constraint.

12. REST API Documentation

Responses are JSON. Errors return { success: false, message }.

Method	Endpoint	Purpose	Auth?	Role	Request / parameter s	Response / result
GET	/health	Check API is running	No	–	–	200
POST	/api/auth/register	Create guest account	No	–	Body: name, email, password	201, token, user; 400 / 409

Method	Endpoint	Purpose	Auth?	Role	Request / parameter s	Response / result s
POST	/api/auth/login	Log in	No	–	Body: email, password	200, token, user; 401
GET	/api/auth/me	Current user	Yes	Any	Bearer token	200, user
GET	/api/rooms	Search active rooms	No	–	Query: type, maxPrice, sort, page, limit	200, data, pagination
GET	/api/rooms/:id	One active room	No	–	Path: id	200; 400 / 404
POST	/api/rooms	Add a room	Yes	Manager	Body: roomNumber, type, pricePerNight, capacity, description?	201; 400 / 401 / 403 / 409
PUT	/api/rooms/:id	Update a room	Yes	Manager	Path: id; Body: any allowed field	200; 400 / 404
DELETE	/api/rooms/:id	Deactivate a room	Yes	Manager	Path: id	200; 409 if confirmed bookings
POST	/api/bookings	Create a booking	Yes	Any	Body: roomId,	201 with totalPri

Method	Endpoint	Purpose	Auth?	Role	Request / parameter s	Response / result
					checkIn, checkOut	ce; 400 / 404 / 409
GET	/api/bookings/my	Own bookings	Yes	Any	Query: status, page, limit	200, data, paginatio on
GET	/api/bookings	List bookings	Yes	Manager: all; guest: own	Query: status, room, guest, page, limit	200, data, paginatio on
GET	/api/bookings/:id	One booking	Yes	Owner or Manager	Path: id	200; 403 / 404
PATCH	/api/bookings/:id/cancel	Cancel a booking	Yes	Owner or Manager	Path: id	200; 400 if already cancelled
DELETE	/api/bookings/:id	Alias of cancel	Yes	Owner or Manager	Path: id	Same as cancel

There are no separate manager-only booking URLs; the controllers apply the manager difference based on role.

13. Frontend and User Flow

The frontend is a React single-page application. `App.jsx` defines the routes and wraps the app in `AuthProvider`.

Route	Page	Access
/	Home page with a link to the rooms	Public

Route	Page	Access
/rooms	Room listing with filter bar and pagination (6 per page)	Public
/rooms/:id	Room details and “Reserve This Room” button	Public to view; login to book
/login, /register	Login and registration	Public
/my-bookings	The user's bookings with a cancel button	Logged-in users
/manager	Manager dashboard	Manager only

- **Search and filter:** SearchFilterBar has category, maximum rate and sort controls; a change resets to page 1 and reloads. Previous / Next use the API's pagination data.
- **Booking form:** BookingModal takes check-in and check-out, shows nights and an estimated total, and sends the request. Server messages, such as an overlap error, appear in the modal. The real validation is on the server.
- **Manager dashboard:** a Room Inventory tab (list, add form, Deactivate button) and a Guest Reservations tab (all bookings). It has no room edit form and no cancel button; those exist in the API.
- **Protected routes:** ProtectedRoute sends visitors who are not logged in to /login, and non-managers away from /manager.
- **API service:** services/api.js has one request function that adds the Bearer token, calls fetch and throws the server's message on failure. A 401 clears the session.
- **Authentication state:** AuthContext keeps user, token and isManager with login, register and logout. The token is stored in localStorage, and /api/auth/me confirms it on page load.
- **Connection to the backend:** in development Vite (port 3000) proxies /api to the backend (port 5001); CORS uses CLIENT_ORIGIN.

Guest flow: register or log in → view rooms → filter and sort → open room details → choose dates → booking request → backend validates dates and checks availability → booking saved → view it in My Bookings (and cancel if needed). A visitor who is not logged in is sent to the login page when booking.

Manager flow: log in (redirected to /manager) → add or deactivate rooms → review all reservations. Price changes and cancelling a guest's booking are done through the API.

14. Validation, Error Handling and Security

Area	What the code does
Required fields	Register, login, create room and create booking each check their required fields (400)
Dates	Valid format; check-in not in the past; check-out after check-in; the schema repeats the last rule
Rooms	Type must be Standard, Deluxe or Suite; price at least 1; capacity a whole number; room number unique; bad type, maxPrice, sort, page or limit return 400
Users	Email format and uniqueness; password at least 8 characters
Bookings	Room must exist and be active; overlap returns 409; only the owner or a manager can view or cancel (403); a cancelled booking cannot be cancelled again
Authentication errors	Missing, invalid or expired token, or a deleted user, returns 401
Authorization errors	A guest calling a manager route returns 403
Invalid ids / routes	Malformed id returns 400; unknown id 404; unknown route returns a JSON 404

Error handling. Controllers use `asyncHandler`, so errors reach `middleware/errorHandler.js`. Expected errors are raised with `AppError(message, status)`. The handler converts Mongoose cast and validation errors (400), duplicate keys (409) and JWT errors (401) into JSON messages. In production, 500 errors show a

generic message and stack traces appear only in development. The frontend displays the server message in an alert box.

Security measures

- Passwords are hashed with `bcryptjs` and never returned.
- JWT protects routes, and the user is re-checked in the database on each request.
- `restrictTo('manager')` guards room create, update and deactivate; public registration cannot create a manager.
- The database URI, JWT secret, token lifetime, port and allowed origin come from `.env`; `.env.example` lists names only. No secret values appear in this report.
- The server sets booking price and status; JSON bodies are limited to 10 kb; room create and update accept only whitelisted fields.
- Frontend route guards are a convenience; the real enforcement is on the server.

Known limitations

- `signToken` in `authController.js` falls back to a built-in default secret if `JWT_SECRET` is missing; it should be removed.
- If `CLIENT_ORIGIN` is not set, CORS allows every origin.
- The token is kept in `localStorage`, and login has no rate limiting.

15. Testing

The ZIP has no saved test report, so no pass or fail results are claimed here. Run the tests and add the outcome if required.

Item	What it covers
<code>tests/health.test.js</code> (Jest, Supertest)	<code>/health</code> returns 200; unknown route returns 404
<code>tests/rooms.test.js</code> (Jest, in-memory MongoDB)	About 35 cases: active-only listing, filters, both sort directions, pagination and limit cap, invalid inputs (400), single room (200 / 400 / 404), create / update / deactivate with 401, 403 and manager success, duplicate room number (409)
<code>tests/verifyBackend.js</code> (run with <code>node</code> , uses <code>MONGO_URI</code>)	About 30 integration checks: registration, role escalation prevention, login, manager login, room RBAC, search, date validation,

Item	What it covers
	valid booking, overlap rejection, same-day turnover, booking access control, cancellation, re-booking a cancelled interval, room deactivation guard. Test data is prefixed and cleaned up
postman/ (three collections)	The Complete collection has six folders: health, authentication, room management, room search, bookings, and error cases (overlap 409, invalid range 400, past date 400). Phase2 covers rooms; Phase3 covers auth and bookings

The frontend has no automated tests. Run the Jest tests with `npm test`.

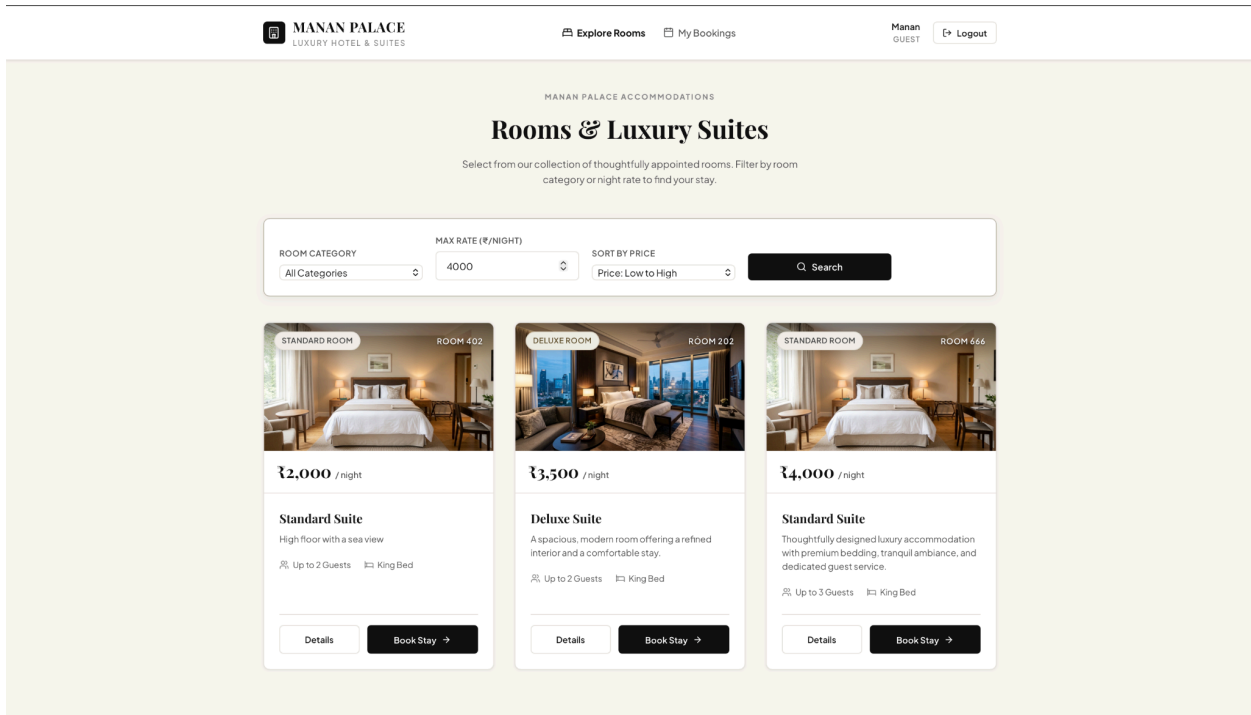
16. Case Study Requirement Mapping

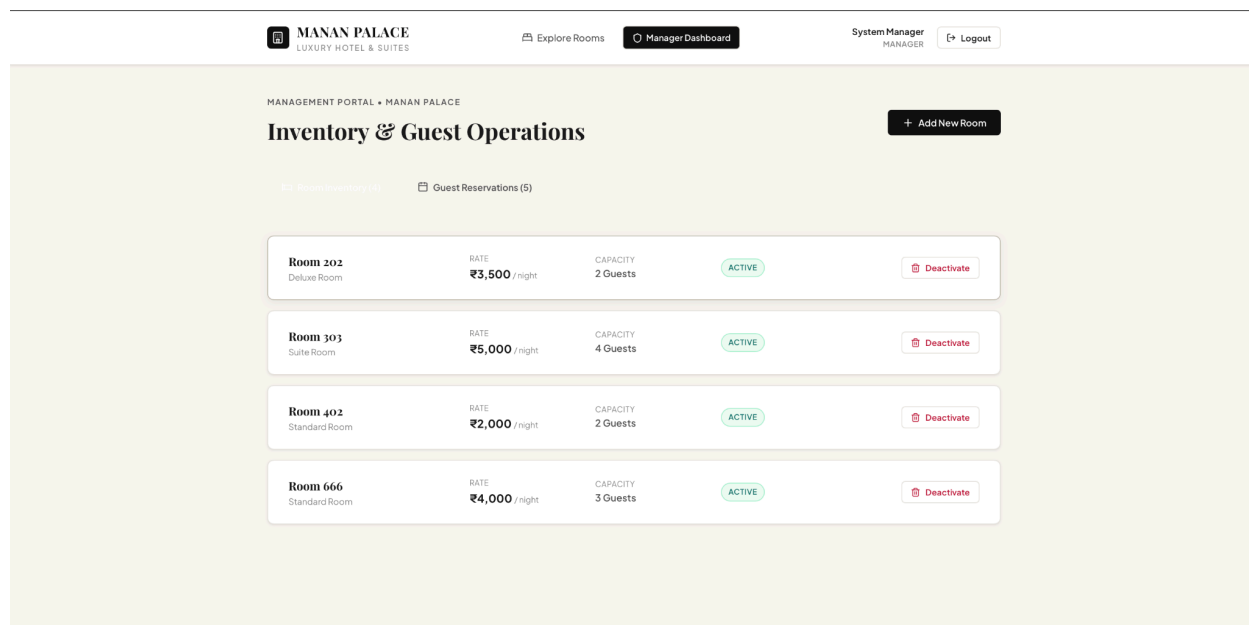
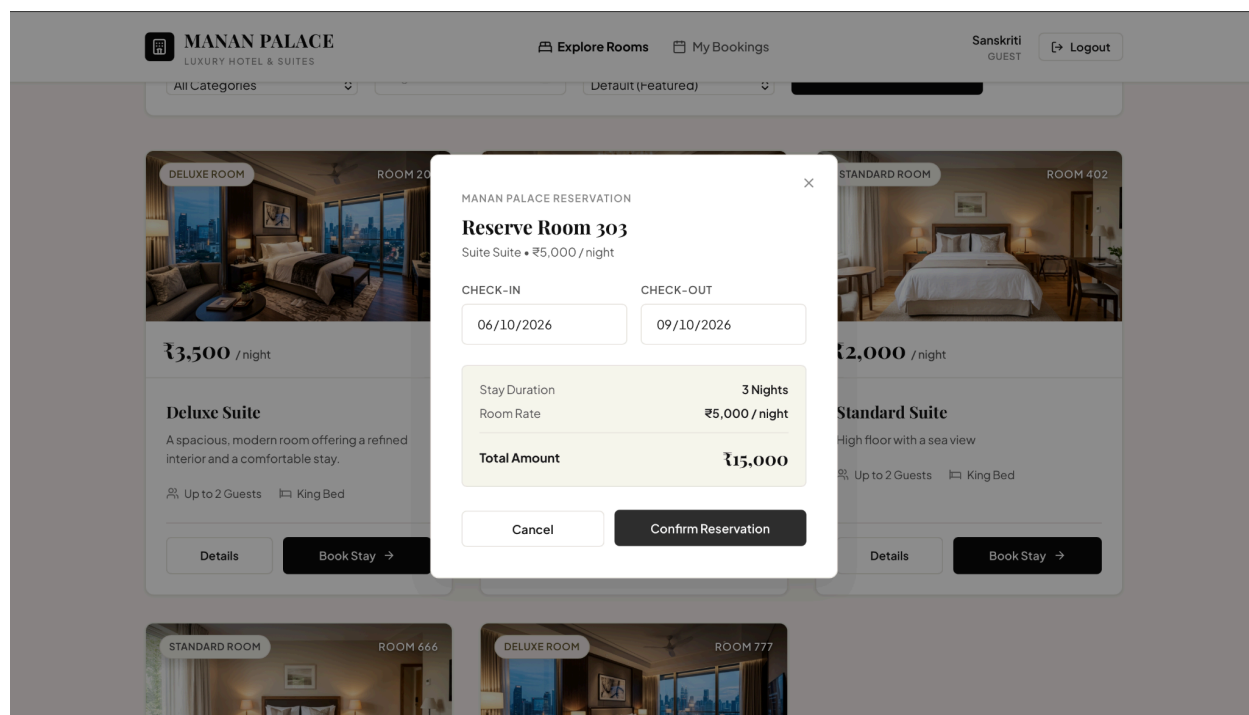
Case Study Requirement	Implementation in Project	Status
Guests search room availability by date	Guests can browse and filter rooms. Availability by date is enforced when booking (<code>POST /api/bookings</code>). The room search does not accept check-in / check-out dates, and the detail page's "Available" label is fixed text, not a live check	Partial
Guests book rooms online	<code>POST /api/bookings</code> , booking modal, My Bookings page	Done
Manager manages room inventory and pricing	<code>POST, PUT, DELETE /api/rooms</code> ; dashboard supports add and	Done (edit form only in API)

Case Study Requirement	Implementation in Project	Status
	deactivate; price update through the API	
Filtering by room type and price, with pagination	type, maxPrice, sort, page, limit on GET /api/rooms; filter bar and pagination controls in the UI	Done
A room cannot be double-booked for the same dates	Booking.hasOverlap inside a transaction; 409 on conflict; cancelled bookings excluded	Done
Objective 1: Room and Booking schemas in Mongoose	models/Room.js, models/Booking.js (plus Guest.js)	Done
Objective 2: CRUD for room inventory	roomController.js; delete is a soft deactivate	Done
Objective 3: Filtering, sorting, pagination	getRooms in roomController.js	Done
Objective 4: Date-based availability check before booking	createBooking and hasOverlap	Done
Objective 5: Role-based authorization for manager routes	protect, restrictTo('manager') in middleware/auth.js	Done
Room schema with type, pricePerNight, capacity	Also roomNumber, description, isActive	Done
Booking schema referencing Room and Guest with checkIn / checkOut	room and guest ObjectId references; also status and totalPrice	Done

Case Study Requirement	Implementation in Project	Status
Check-in / check-out date validation	Controller checks and schema validator	Done
MongoDB Atlas connection and .env configuration	config/db.js uses MONGO_URI; .env.example provided	Done
Postman / Thunder Client collection	Three Postman collections	Done
React frontend: search, booking form, manager panel, login / register, list, add form, details	RoomsPage, BookingModal, ManagerDashboard, LoginPage / RegisterPage, RoomDetailPage. The update form is not in the UI	Mostly done
Frontend calls REST APIs with Fetch / Axios; logic stays in backend	services/api.js (Fetch); validation and overlap logic are in the backend	Done
Deployment note (Render / Railway, Vercel / Netlify)	The backend README has deployment steps; the project does not show a deployed link	Not shown

17. Screenshots





18. Challenges, Future Scope and Conclusion

Challenges and solutions

Challenge	How the project handles it
Preventing overlapping bookings	One overlap query on confirmed bookings, with an index; same-day turnover allowed
Concurrent requests	The check and save run in a Mongoose transaction, with a fallback when transactions are unavailable
Keeping booking history	Rooms are deactivated, not deleted; a room with confirmed bookings cannot be deactivated
Safe authentication	Hashed passwords, JWT with a database re-check, role forced to guest on registration
Connecting frontend and backend	Vite proxy, one request helper that sends the token, CORS through CLIENT_ORIGIN
Consistent errors and configuration	Central error handler with <code>AppError</code> ; <code>config/db.js</code> reads <code>MONGO_URI</code> from <code>.env</code> and stops with a clear message if it is missing

Future scope (possible improvements, not current features): a date filter on room search; a room edit form and booking cancel option in the manager dashboard; a database-level guarantee against overlapping bookings; online payment; email confirmation; manager reports; deployment to a cloud platform.

Conclusion. The Hotel Room Booking System is a working full-stack application for Case Study 126, built with Node.js, Express.js, Mongoose and MongoDB Atlas on the backend and React with Vite and Fetch on the frontend. Guests can register, search rooms with type and price filters, sorting and pagination, book rooms and manage their bookings. Managers can add, update (through the API) and deactivate rooms and view all reservations.

The project meets the main requirements of the case study. The one gap is that room search does not take dates; availability is checked when the booking is made. That check, which rejects any booking overlapping a confirmed booking for the same room,

together with role-based access using JWT and `restrictTo('manager')`, protects the inventory and prevents overlapping bookings.

19. References

1. Node.js Documentation. <https://nodejs.org/docs/latest/api/>
2. Express.js Documentation. <https://expressjs.com/>
3. MongoDB Documentation (including Atlas). <https://www.mongodb.com/docs/>
4. Mongoose Documentation. <https://mongoosejs.com/docs/>
5. React Documentation. <https://react.dev/>
6. React Router Documentation. <https://reactrouter.com/>
7. Vite Documentation. <https://vite.dev/guide/>
8. JSON Web Token introduction. <https://jwt.io/introduction>
9. bcryptjs (npm package). <https://www.npmjs.com/package/bcryptjs>
10. Jest Documentation. <https://jestjs.io/docs/getting-started>
11. Postman Learning Center. <https://learning.postman.com/docs/>